The Neuron Migration

Rebuilding 30 years of insurance logic as code an auditor can check

Team NAMA: Mohammad Javad Amin, Nava Biglar, Malihe Ghasemian, Emad Aghajani
Lifeware Challenge, Neural Wave AI Hackathon 2026 USI Lugano

In one paragraph. NeuralWave Life's legacy system Neuron has no source code and no reliable documentation, yet the new system must reproduce its numbers to the cent. We found Neuron's rules for all tasks from the data alone and wrote them as about 2,700 lines (tools included) of plain Python with exact decimals. On the training policies every value that is still different from Neuron's is a value someone changed by hand: we recalculate 5,024,847 fund bookings and all of them match except three manual edits. The surprise task t7 (tax audit) came without any target values; we solved it with the same method, using Neuron's own tax bookings as labels.

1 Approach

1.1 AI finds the rules, plain code applies them

A model that predicts a surrender value is never exact and cannot be audited, so no model runs inside our submission. Machine learning and search are only used to *find* rules; every rule is then written as a short, deterministic function (one module per task, `Decimal` arithmetic, no hard-coded values of this export).

Data processing. One loader decodes all 27 tables. It reads the data dictionary `NRDICT` first and decodes every column by its type (padded text, implied decimals, `CYYMMDD` dates, timestamps), so there is no code per table. Decoding everything this way already surfaced the data problems: impossible birth dates (31 November), a code list with a shifted column, lowercase currency codes, a smoker flag `J/N` instead of `Y/N`.

1.2 One discovery loop with generic tools

For every target we run the same loop: write the simplest rule, mark the values it gets wrong, ask the tools *why*, turn the answer into code, and repeat until only manual corrections are left. None of the tools knows anything about insurance:

Tool	Question it answers	Example of what it found
Rule finder (shallow decision tree)	Under which conditions is the rule wrong?	due dates drift for month-end starts before 2007
Change-point finder	On which day did the behaviour change?	the COBOL → Java conversion on 2007-08-11
Residual analysis	What do the differences say?	ratios of 31/360 and 28/360: COBOL charged by the day
Convention search	Which convention is it?	order of the charges; Actual/360 with cut-off rounding
Weak labels	No targets: does a rule book what Neuron booked?	Swiss withholding needs an objection and > CHF 5,000
Learned calendar	Which days are business days?	the Ticino holidays, learned from the fund price days

Notebook 06 runs each tool on its own and shows it rediscovering one of our rules. For the fund accounting (t3, t4) we added one more idea: *teacher forcing*. We take the holdings before each booking from Neuron's diagnostic bookings, so that every booking checks one formula in isolation instead of carrying earlier errors forward.

1.3 What the loop found

The central finding is that Neuron is really two systems. Until the conversion on 2007-08-11 the COBOL system cut off every amount, computed the age as days since birth / 365 (ignoring leap days), charged the management fee by actual days / 360 and let due dates drift (31 → 30 → 28). Neuron NG rounds half up, takes the age at the nearest birthday, charges a twelfth of the yearly rate and anchors the due dates. The accounting also has operational rules nobody wrote down: the charge run at 01:30 only sees units traded before that day, each fund buys on its own first day with a price, rounding remainders go to the largest position, and the Italian stamp duty is re-booked when a tax rate is corrected afterwards. The extended report lists every rule with its evidence.

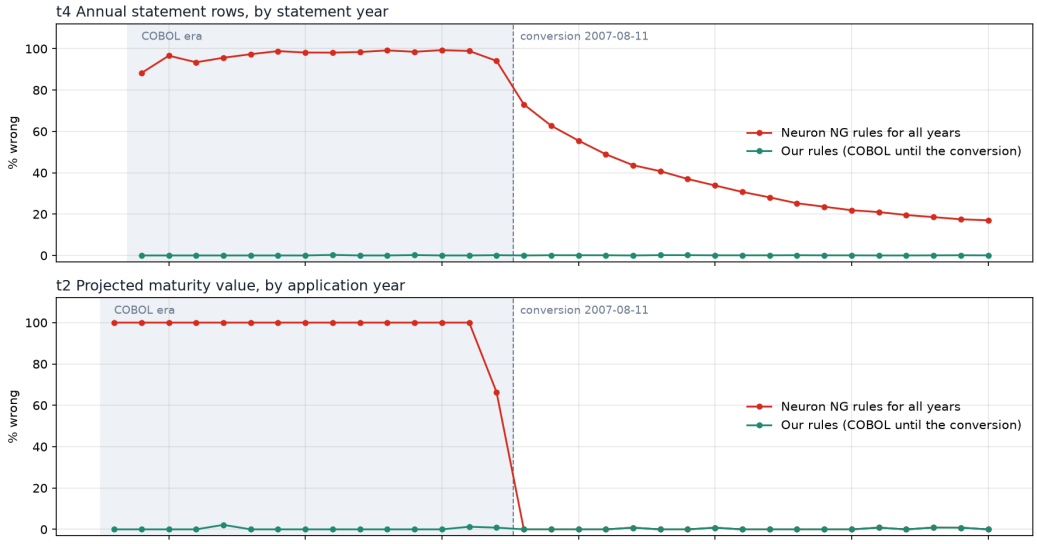
t7 without targets. The organisers could not give us t7 targets. We used three other sources of evidence: Neuron's tax bookings (which tax was booked for which payout and year, without amounts) as weak labels, the two examples of the task text as exact anchors for a convention search, and a consistency check that every tax Neuron booked gets an amount. Both examples come out to the cent (5,628.16 and 616.11).

2 Results

Official scoring on the 3,019 training policies:

Task	Exact rows	Accuracy
t0 Commission	225,848 / 225,858	99.996 %
t1 Next due dates	15,028 / 15,028	100 %
t2 Projected maturity value	3,011 / 3,019	99.735 %
t3 Surrender value	6,137 / 6,153	99.740 %
t4 Annual statement	34,124 / 34,145	99.939 %

- **Every remaining miss is a manual correction**: whole-franc differences while the values around them (yearly projections, policy values, other bookings) are exact. The official scoring does not count them.
- **t5**: our outlier detection reports exactly these 55 values, including all 38 manual corrections listed for these policies in `NREXCP` and 17 that nobody had listed.
- **t6**: the figure shows why the era matters. With the Neuron NG rules alone, 97 % of the statement rows of the COBOL era are wrong, and the error fades only slowly after 2007 (still 17 % in 2025), because old policies carry the wrong balance forward. Our rules stay at 0 %.
- **Runtime and code**: t0–t5 run in about 1.5 minutes (t7 adds one), with a radon maintainability index of 68. The first version took 6 minutes; reading policies as tuples instead of pandas rows made it 5× faster.



Share of wrong values per year with the Neuron NG rules for all years (red) and with our rules (green), t4 statements and t2 offers; the COBOL era is shaded.

3 Challenges and what did not work

- **Wrong first hypotheses**, each rejected by an exact test: holdings in booking order (the charge run happens before same-day trades), the FX rate of the trade day for premiums (it is the day before), growth compounded as $(1+r)^{1/12}$ and applied before the charges, rate / 12 in the COBOL era, the first publication of restated prices, rounding remainders on the first or last fund, anchored due dates for COBOL policies.
- **A test that lied**: we first compared amounts with `np.isclose`, whose relative tolerance hid one-cent errors on large values (99.4 % looked like progress; the official score was 95.9 %). Since then every check compares to the cent.
- **Ambiguous evidence**: the age that COBOL used fitted 97 % of the offers with the nearest birthday. Only testing single data changes (one year older) and then several age formulas isolated days / 365.
- **No targets for t7**: the open points are the Swiss and Italian remittance dates (30 days after the month end, the 16th of the next month), the share of the year in the stamp duty booked at a payout, and the German partial exemption from 2018. They follow the law and the data structure but could not be checked to the cent.
- **Trade-offs**: splitting modules raised the maintainability index, merging small files lowered it; we measured each change with radon instead of guessing.

4 AI agents and LLM usage

We worked with Claude Code, Anthropic's agentic coding assistant (Claude Opus models), throughout the hackathon. It wrote most of the code, the notebooks, the first drafts of this report and the slides, ran the experiments in a local Python environment and proposed hypotheses from the residual patterns (for example that a ratio of 1.033 means 31 days / 360). The team set the direction, decided which ideas to pursue, reviewed the results and checked everything with the official scorer; every rule in the code is backed by an exact test on the data. For t7 the task text asks for custom or open-weights models because of the tax data; we asked the organisers, who confirmed that a coding assistant is allowed. No language model and no machine-learning model runs inside the submission: decision trees and searches are only used in the discovery tools, and the submitted functions are deterministic Python.